

Software Test Plan

Vehicle Parking System

Group 14 — Problem Statement No. 10

Project	Vehicle Parking System (MERN Stack)
Document	Software Test Plan (STP)
Date	29-09-2026
Reference	Vehicle Parking System — Software Requirements Specification
Authors	Siddhesh Dattatray Koditkar — PES2UG24CS502 — Project Lead & Development Lead Sumukh M Gowda — PES2UG24CS537 — Backend & Integration Lead Sparsha Arun — PES2UG24CS513 — Frontend & Test Automation Lead Sinchana Kulkarni — PES2UG24CS504 — Security & Release Lead

Table of Contents

1. Introduction
2. Test Items
3. Features to be Tested
4. Features Not to be Tested
5. Test Approach / Strategy
- 5.1 Security Validation
6. Test Environment
7. Test Schedule
8. Test Deliverables
9. Roles and Responsibilities
10. Risks and Mitigation
11. Assumptions & Dependencies
12. Suspension & Resumption Criteria
13. Test Case Management & Traceability
14. Test Metrics & Reporting
15. Approvals

1. Introduction

1.1 Purpose

This Software Test Plan (STP) defines the objectives, scope, strategy, resources, schedule, and responsibilities for testing the Vehicle Parking System (VPS) v1.0. It exists to verify that the functional, non-functional, and security requirements defined in the VPS Software Requirements Specification (SRS) v1.0 are correctly implemented before submission.

1.2 Scope

Testing covers: authentication and role-based access control, vehicle entry and concurrency-safe spot allocation across multiple gates, real-time lot visualization, exit processing and tiered fare calculation, overstay and lost-ticket escalation, admin analytics and audit oversight, and the security controls enforcing role separation. For the purpose of defining test boundaries, this plan assumes that — were the system deployed at a real facility — physical gate hardware and payment processing would be provided by certified third-party vendors; verifying that hardware and that payment processor is therefore outside this plan's scope (see Section 4).

1.3 References

- Vehicle Parking System — Software Requirements Specification (SRS), v1.0
- Software Engineering course project guidelines (Group 14, Problem Statement No. 10)

1.4 Definitions, Acronyms and Abbreviations

- VPS — Vehicle Parking System
- STP — Software Test Plan
- FR / NFR — Functional Requirement / Non-Functional Requirement
- RBAC — Role-Based Access Control
- JWT — JSON Web Token
- FCFS — First-Come, First-Served
- TC — Test Case
- RTM — Requirements Traceability Matrix

2. Test Items

- Authentication & Access Control module
- Vehicle Entry & Spot Allocation module (concurrency-critical)
- Real-Time Lot Visualization module (Socket.io broadcast)
- Vehicle Exit & Fare Calculation module
- Overstay & Lost-Ticket Escalation module
- Admin Analytics & Oversight module
- Audit Logging & Data Integrity module

The Simulation Console is explicitly excluded — see Section 4.

3. Features to be Tested

Features mapped to VPS SRS requirement IDs:

- VPS-F-001..005 — Authentication & Access Control
- VPS-F-010..016 — Vehicle Entry & Spot Allocation
- VPS-F-020..023 — Real-Time Lot Visualization

- VPS-F-030..037 — Vehicle Exit & Fare Calculation
- VPS-F-040..043 — Overstay & Lost-Ticket Escalation
- VPS-F-050..056 — Admin Analytics & Oversight
- VPS-F-060..062 — Audit Logging & Data Integrity
- VPS-NF-001..007 — Non-Functional Requirements (performance, concurrency, availability, usability, maintainability, scalability, accuracy)
- PRJ-SR-001..006 — Security Requirements

4. Features Not to be Tested

- Physical barrier/gate hardware and sensors — assumed vendor-supplied and vendor-tested in a real deployment.
- Payment-processing hardware/terminal and payment-gateway integration — assumed a certified third-party processor in a real deployment; the VPS software only calculates and displays the fare.
- Underlying platform internals (Node.js runtime, MongoDB engine, Socket.io library, browser engine) — assumed stable and tested upstream.
- Hosting/network infrastructure uptime — assumed managed by the hosting provider.
- VPS-DEV-001..003 (Simulation Console) — a developer/demo-only utility, explicitly outside the deployed system's role model per the SRS; not covered by this plan's acceptance criteria.

5. Test Approach / Strategy

5.1 Test Levels

- Unit — individual functions in isolation (fare-tier calculation, JWT validation, allocation query construction).
- Integration — modules working together (allocation endpoint through to the MongoDB write and the Socket.io broadcast it triggers). Catches contract mismatches between pieces built independently.
- System — full entry-to-exit flow exercised through the real UI across multiple simulated gates (browser tabs). Validates the actual demo scenario end to end.
- Acceptance — each SRS requirement's stated Acceptance Criteria is checked directly and marked pass/fail. Doubles as proof of completion against the SRS.

5.2 Test Types

- Functional — verifies each FR/NFR behaves exactly as specified in the SRS.
- Performance — concurrency/load testing: fire a batch of simultaneous allocation requests across multiple gates and confirm zero duplicate allocations and broadcast latency within the NFR target (VPS-NF-001, VPS-NF-002).
- Regression — re-run the functional suite after significant changes; low additional cost since it is the same automated suite re-executed.
- Usability — one informal walkthrough of the operator/admin UI against the 3-interaction NFR target (VPS-NF-004); no formal usability lab.

5.3 Entry Criteria

- The feature branch being tested is merged to main.
- A local development environment (Node.js + MongoDB) is running and reachable.
- Seed/test data (dummy vehicles, gate-operator and admin accounts) is available.

5.4 Exit Criteria

- All High-priority test cases (per the SRS RTM) have been executed and pass.

- Zero critical defects remain open (duplicate spot allocation, incorrect fare calculation, cross-gate or cross-role access).
- No High-priority item in the RTM (Section 13) is left unresolved.

5.1 Security Validation

Mapped directly to the SRS's security requirements (PRJ-SR-001..006):

- Verify a valid JWT is required on every protected route, and requests without one are rejected (PRJ-SR-002).
- Verify passwords are stored only as salted hashes and never logged in plaintext (PRJ-SR-003).
- Verify gate-scoped operator credentials cannot perform actions against a gate they are not bound to, even via direct API calls (PRJ-SR-004).
- Verify Gate Operator accounts cannot reach Admin-only endpoints (analytics, raw records, force-release) (PRJ-SR-005).
- Verify every state-changing action (allocation, release, force-release, penalty) produces a corresponding audit-log entry (PRJ-SR-006).
- Verify TLS is enforced for any non-local deployment configuration (PRJ-SR-001).

6. Test Environment

6.1 Hardware

No specialized hardware required.

6.2 Software

- Node.js 24 LTS runtime and npm
- MongoDB Community Server (local instance)
- VPS application under test (backend + frontend)

6.3 Tools

- Jest — test runner and assertion library for backend and frontend unit tests
- Supertest — HTTP-level integration testing of Express API endpoints, run inside Jest
- React Testing Library — component-level frontend tests, run inside Jest
- k6 — concurrency/load testing for the multi-gate allocation scenario
- Git / GitHub — version control and collaboration
- Postman (optional) — manual/exploratory API testing during development

6.4 Test Data

A seed script generates dummy vehicles (plate numbers, 2W/4W split), four operator accounts (Entry-1, Entry-2, Exit-1, Exit-2), and one admin account, so the same starting state can be reproduced on any team member's machine.

7. Test Schedule

Testing is carried out alongside development each week, not concentrated at the end. Four-week plan starting Friday, 2 October 2026:

Week	Dates	Build Focus	Test Focus
1	Oct 2 – Oct 8	Scaffolding: Express/Mongo/React skeleton, Socket.io wiring, gate-scoped JWT auth	Unit tests for auth as it's built; Jest and k6 environment set up
2	Oct 9 – Oct 15	Entry/allocation flow, atomic concurrency-safe allocation, exit/fare calculation	Integration tests for allocation and fare — highest-risk logic, tested as it's written
3	Oct 16 – Oct 22	Real-time sync across tabs, admin dashboard/analytics/audit log, overstay/lost-ticket flow, Simulation Console	System tests (full entry-to-exit across simulated gates); security validation pass
4	Oct 23 – Oct 29	Bug fixes, polish, demo dry-run	k6 load/concurrency test executed formally; regression pass; finalize execution log and test summary report

8. Test Deliverables

- Software Test Plan — this document, finalized and signed off.
- Test Execution Summary / Log — a record of each test case's pass/fail result, produced at the end of Week 4.

9. Roles and Responsibilities

All members contribute to development and testing across modules in practice.

Role	Name	Primary Responsibility
Project Lead & Development Lead	Siddhesh Dattatray Koditkar	Coordinates the test plan and schedule, tracks exit-criteria readiness, and has final say on release/submission timing.
Backend & Integration Lead	Sumukh M Gowda	Owns backend and API-level testing — allocation logic, fare calculation, Integration-level test cases.
Frontend & Test Automation Lead	Sparsha Arun	Owns frontend/component testing and the Jest/RTL automation setup — System-level and UI test cases.
Security & Release Lead	Sinchana Kulkarni	Owns security validation (5.1) and the concurrency/load test (k6), and prepares the final test execution log.

10. Risks and Mitigation

Risk	Mitigation
Concurrency bugs in spot allocation only surface under simultaneous multi-gate load, not casual manual testing.	Dedicated k6 load test in Week 4, not just manual click-through testing.
Socket.io state can drift between browser tabs if a client reconnects or misses an event.	Explicit integration tests covering reconnect/refresh scenarios, not just the happy path.
The 4-week timeline compresses available test time if development runs long.	Tests are written alongside each week's features (see Section 7), not deferred to Week 4.

11. Assumptions & Dependencies

- All members have a working local Node.js + MongoDB development environment.
- Test data (dummy vehicles, gate/admin accounts) is generated by the team's own seed script, not sourced externally.

12. Suspension & Resumption Criteria

Suspend testing if:

- The core allocation or fare-calculation logic is broken, since most other test cases depend on it.

- The local test environment (Node/Mongo) is unavailable for an extended period.

Resume testing once the blocking defect is fixed and merged, or the environment is restored.

13. Test Case Management & Traceability

Test Case IDs were assigned when the SRS was written. Status: N (Not started), P (In progress), A (Achieved/passed) — all begin as N.

Req ID	Test Case(s)	Level	Owner	Status
VPS-F-001	TC-Auth-01	Unit	Security & Release Lead	N
VPS-F-005	TC-Auth-05	Unit	Security & Release Lead	N
VPS-F-012	TC-Entry-03	Integration / Performance	Backend & Integration Lead	N
VPS-F-013	TC-Entry-04	Integration	Backend & Integration Lead	N
VPS-F-015	TC-Entry-06	Unit	Backend & Integration Lead	N
VPS-F-021	TC-Viz-02	Integration	Frontend & Test Automation Lead	N
VPS-F-033	TC-Exit-04	Unit	Backend & Integration Lead	N
VPS-F-035	TC-Exit-06	Integration	Backend & Integration Lead	N
VPS-F-042	TC-Over-03	System / Security	Security & Release Lead	N
VPS-F-050	TC-Admin-01	Security	Security & Release Lead	N
VPS-F-060	TC-Audit-01	Integration	Backend & Integration Lead	N
PRJ-SR-003	TC-Sec-03	Security	Security & Release Lead	N
PRJ-SR-004	TC-Sec-04	Security	Security & Release Lead	N
VPS-NF-002	TC-Rel-01	Performance	Security & Release Lead	N
VPS-NF-001	TC-Perf-01	Performance	Frontend & Test Automation Lead	N

This is a representative subset; the full requirement list (51 testable IDs: 38 functional + 7 non-functional + 6 security) follows the same mapping pattern against the SRS's own Test Ref column.

14. Test Metrics & Reporting

Metrics collected:

- % of test cases executed
- % passed / failed
- Open defect count by severity
- Requirement coverage (X of 51 testable requirement IDs with a passing test case)

Reporting:

- Informal weekly team sync on testing status (Weeks 1–4)
- Final Test Summary Report at the end of Week 4